

Joc del Pingu — Guia tècnica per a desenvolupadors

Estructura, tecnologies i decisions de disseny

Grup DW25-26 GR06 — Curs 2025/2026

Índex

- 1. Visió general del projecte
- 2. Tecnologies utilitzades
- 3. Estructura del projecte
- 4. Organització del codi (classes principals)
- 5. Funcionament general (flux de l'aplicació)
- 6. Relacions entre classes
- 7. Decisions de disseny rellevants
- 8. Instruccions per compilar i executar

1. Visió general del projecte

Joc del Pingu és un joc de taula multi-jugador (1 a 4 participants) inspirat en l'oca, ambientat en un escenari àrtic. Cada jugador controla un pingüí que avança pel tauler mitjançant tirades de dau, recull objectes (boles de neu, peixos, daus alternatius) i interactua amb caselles especials (forats de gel, trineus, esdeveniments, terra trencat). Opcionalment es pot habilitar una **foca** controlada per IA com a antagonista.

L'aplicació segueix una arquitectura **MVC** amb tres capes ben diferenciades (Model / View / Controller) i utilitza **JavaFX** per a la interfície gràfica. La persistència es fa contra una BBDD Oracle.

2. Tecnologies utilitzades

Tecnologia	Versió	Propòsit
Java	17+ (mòdul Java 9+)	Llenguatge principal
JavaFX	21+	Interfície gràfica (FXML, CSS, Canvas, Media)
Oracle JDBC	ojdbc8/11	Connexió a la BBDD Oracle
SnakeYAML	1.x / 2.x	Serialització de l'estat de la partida
JCE (Java Cryptography)	Estàndard JDK	Encriptació AES-128 de contrasenyes i partides guardades
Eclipse IDE	2024+	IDE de desenvolupament (workspace inclòs)
FXML + CSS	—	Definició declarativa de pantalles i estils

[CAPTURA 2.1] Pantalla d'Eclipse mostrant el classpath del projecte amb javafx-lib, ojdbc, snakeyaml.

3. Estructura del projecte

El projecte segueix una estructura per *paquets* que separa molt clarament les responsabilitats:

```
JocDelPingu/
├── src/
│   ├── controller/
│   │   ├── main/
│   │   │   └── MainMenu.java           ← Entry point JavaFX
│   │   └── ui/
│   │       ├── MainMenuController.java
│   │       ├── PlayerSetupController.java
│   │       ├── GameBoardController.java
│   │       └── PlayerStatsController.java
│   └── model/
│       ├── board/                     ← Tauler i caselles
│       │   ├── Board.java
│       │   ├── Square.java
│       │   ├── SquareType.java
│       │   ├── EventManager.java
│       │   └── squares/               ← Subtipus de casella
│       │       ├── S_Normal, S_Start, S_End
│       │       ├── S_IceHole, S_Sled
│       │       └── S_Bear, S_Event, S_BrokenFloor
│       ├── config/
│       │   ├── GameSetupConfig.java   ← Hand-off entre controladors
│       │   ├── LangConfig.java        ← i18n
│       │   ├── Lang.java
│       │   └── CryptoUtil.java        ← AES-128
│       ├── db/
│       │   └── BBDD.java              ← Facade JDBC
│       ├── entity/
│       │   ├── Entity.java (abstract)
│       │   ├── EntityType.java
│       │   ├── Player.java
│       │   └── Seal.java
│       ├── game/
│       │   ├── Game.java              ← Aggregate state
│       │   ├── GameManager.java       ← Orquestrador
│       │   ├── BoardManager.java
│       │   ├── PlayerManager.java
│       │   ├── TurnController.java
│       │   ├── SaveLoadService.java   ← Persistència
│       │   ├── SoundManager.java
│       │   └── ActionResult.java
│       └── item/                      ← Inventari i objectes
│           ├── GameObject.java
│           ├── Inventory.java
│           ├── ObjectType.java
│           └── objects/
│               ├── Dice, Fish, SnowBall
```



3.1. Paquets clau

Paquet	Responsabilitat
<code>controller.main</code>	Punt d'entrada de l'aplicació JavaFX (MainMenu)
<code>controller.ui</code>	Controladors FXML que connecten les pantalles amb la lògica
<code>model.board</code>	Representació del tauler i les seves caselles
<code>model.entity</code>	Entitats del joc: jugador, foca
<code>model.item</code>	Objectes i inventari (peixos, boles de neu, daus)
<code>model.game</code>	Orquestració de la partida (torns, accions, persistència)
<code>model.db</code>	Capa d'accés a BBDD Oracle (JDBC)
<code>model.config</code>	Configuració global: idioma, encriptació, setup compartit
<code>view.fxml</code>	Definicions declaratives de les pantalles
<code>view.ui</code>	Panells auxiliars (per ex. el BBDDPanel de proves CRUD)
<code>assets</code>	Recursos: sprites, fonts, sons, traduccions, CSS

4. Organització del codi (classes principals)

4.1. Capa *Controller*

MainMenu (controller.main)

Punt d'entrada de l'aplicació. Hereta de `javafx.application.Application`. Les seves responsabilitats:

- Carregar les fonts pixel-art abans de qualsevol FXML.

- Carregar l'idioma per defecte (`LangConfig.loadLang()`).
- Construir l'`Stage` principal i mostrar el menú.
- Gestionar el mode fullscreen i la restauració de dimensions.

MainMenuController

Controlador de `mainMenu.fxml`. Gestiona:

- Botons "New Game", "Load Game", "Stats".
- ComboBox de canvi d'idioma en runtime.
- Fons del menú i música de títol (`SoundManager.playTitleMusic()`).
- Validació de contrasenyes en carregar partides multi-jugador.

PlayerSetupController

Controlador de `playerSetup.fxml`. Gestiona:

- Combo per escollir el nombre de jugadors (1-4).
- Per a cada jugador: nom, contrasenya, color (`ColorPicker`), avatar (`FileChooser`).
- Botó "Select Existing Player" → carrega jugadors registrats de la BBDD.
- Validació de contrasenyes contra els hashes encriptats.
- Registre del jugador via `SaveLoadService.registerPlayer()`.

GameBoardController

La classe més gran i complexa del projecte. Controlador de `gameBoard.fxml`. Responsabilitats:

- Renderitzar el tauler (Canvas amb sprites pixel-art) en patró de serp.
- Construir el HUD: retrats, inventaris, indicador de torn.
- Animacions: tirada de dau, moviment, llançament de bola de neu, atac de l'os.
- Loop de torns coordinat amb `TurnController` i `GameManager`.
- Gestió de la foca (torn dedicat, animacions, cooldown).
- Save / Load i seqüència cinematogràfica de victòria.
- Mode **debug** ocult (Ctrl+Shift+D): teletransportar jugadors, forçar tirades, editar inventaris.
- Història d'esdeveniments (botó "📖 History").

PlayerStatsController

Controlador de `playerStats.fxml`. Mostra el rànding global amb medalles per al top-3, basat en `SaveLoadService.getPlayerStats()`.

4.2. Capa Model

Game (model.game)

POJO que agrupa tot l'estat d'una partida (tauler, jugadors, torn actual, guanyador, foca, gameOver). És el que es serialitza a YAML per al sistema de save/load.

GameManager

Orquestrador d'alt nivell. La UI només parla amb aquesta classe. Delega a:

- **BoardManager** → resol l'efecte de cada casella.
- **PlayerManager** → mou els jugadors, combat de boles.
- **TurnController** → rotació de torns.
- **Game** → estat persistent.
- **SaveLoadService** → guardar/carregar.

Board (model.board)

Array fix de **Square** (50 caselles, 10×5). Gestiona dues llistes auxiliars (**IceHole_Array**, **Sled_Array**) per resoldre les caselles encadenades.

Square i subtipus (model.board.squares)

Jerarquia de caselles, una classe per tipus:

- **S_Normal**: casella neutra.
- **S_Start**, **S_End**: entrada i meta.
- **S_IceHole**: forat de gel — retorna al forat anterior.
- **S_Sled**: trineu — avança al següent trineu.
- **S_Bear**: atac d'os → salta el següent torn.
- **S_BrokenFloor**: terra trencat → es converteix en forat.
- **S_Event**: dispara un esdeveniment aleatori (**EventManager**).

Player i Seal (model.entity)

Subclasses de **Entity** (classe abstracta). Player conté:

- Nom, color, contrasenya encriptada, avatarPath.
- Inventory (boles de neu, peixos, daus ràpid/lent).
- Historial d'esdeveniments (cap a 50 últimes accions).

BBDD (model.db)

Facade JDBC. Connecta a Oracle i exposa **insert** / **update** / **delete** / **select** / **print** / **executeInsUpDel**. La connexió no està pool·litzada: cada operació crea i tanca la seva (acceptable per a un joc d'un usuari local).

SaveLoadService (model.game)

Facade per a tota la persistència. Mètodes principals:

- **saveGame()** → YAML + AES + INSERT a SAVED_GAMES (PreparedStatement).
- **loadGame()** → SELECT + Decrypt + parseYAML.
- **registerPlayer()** → MERGE idempotent a ENTITY.
- **verifyPassword()** → SELECT + Decrypt + comparació.
- **getPlayerStats()** → leaderboard ordenat per victòries.
- **recordGameResult()** → INSERT a GAME + UPDATE de comptadors.

CryptoUtil (model.config)

AES-128 en mode ECB+PKCS5 amb clau fixa de 16 bytes (`PinguGameKey1234`). Retorna ciphertext Base64-encoded perquè es pugui guardar en text pla a YAML o a la BBDD. *No és una mesura de seguretat forta* — és obfuscació per al joc local.

LangConfig (model.config)

Sistema d'internacionalització. Carrega un YAML d'idioma (`assets/lang/<codi>.yaml`) i exposa `getLang(Lang)`. Suporta canvi de llengua en runtime mitjançant listeners.

Idiomes disponibles: ar, ca, en, en_es, es, es_ca, ff, fr, jp, pt, ro, ru, uk.

[CAPTURA 4.1] ComboBox del menú principal mostrant la llista d'idiomes desplegada.

4.3. Capa View

Definida exclusivament en **FXML + CSS**. Cada pantalla consisteix en:

- Un fitxer `.fxml` que descriu l'arbre de nodes.
- Una referència al controlador via `fx:controller`.
- Estils centralitzats a `assets/css/style.css`.

[CAPTURA 4.2] Arbre de fitxers FXML a Eclipse (view/fxml/) i mainMenu.fxml obert mostrant l'estructura.

5. Funcionament general (flux de l'aplicació)

5.1. Inicialització

1. `MainMenu.main()` carrega l'idioma per defecte.
2. Es crida `Application.launch()`.
3. `MainMenu.start(Stage)` carrega les fonts pixel-art.
4. Es carrega `mainMenu.fxml` → s'instancia `MainMenuController`.
5. `MainMenuController.initialize()` configura el fons, els botons, l'idioma i la música.

5.2. Flux "New Game"

1. Usuari prem **New Game**.
2. Es carrega `playerSetup.fxml`.
3. L'usuari emplena els camps (nom, contrasenya, color, opcionalment avatar).
4. Pot fer servir **Select Existing Player** → `SaveLoadService.getRegisteredPlayers()`.
5. Al prémer **Start Game**:
 - `verifyPassword()` per a cada jugador existent.
 - `registerPlayer()` (MERGE) per als nous.
 - Els jugadors es guarden a `GameSetupConfig`.
6. Es carrega `gameBoard.fxml` → `GameBoardController.initialize()` crea el `Board`, el `GameManager` i comença la partida.

5.3. Flux "Load Game"

1. Usuari prem **Load Game**.
2. `SaveLoadService.getAllSavedGameIds()` consulta `SAVED_GAMES`.
3. Es mostra un `ChoiceDialog` amb la llista.
4. En seleccionar: `SaveLoadService.loadGame(gameId)` → `SELECT` + `Decrypt` + `parseYAML`.
5. Es demana la contrasenya a cada jugador → `verifyPassword()`.
6. Si tot va bé, es carrega `gameBoard.fxml` i el `GameBoardController` detecta `GameSetupConfig.isLoadedGame() == true` i reconstrueix l'estat.

5.4. Flux d'un torn

1. El jugador actiu prem  **Roll Dice** (o un dau alternatiu del seu inventari).
2. `GameBoardController` anima la tirada → resultat 1-6.
3. El jugador es mou casella per casella (animació).
4. En arribar: `BoardManager` resol l'efecte segons `SquareType`.
5. Si hi ha altres jugadors a la mateixa casella, es pot iniciar una guerra de boles de neu.
6. Si la foca està habilitada i comparteix casella amb un jugador, atac de la foca (perd la meitat de l'inventari).
7. `TurnController.nextTurn()` passa al següent jugador (saltant els que tenen `skipNextTurn`).
8. Si algú arriba a la casella `END` → seqüència de victòria + `recordGameResult()`.

[CAPTURA 5.1] Tauler en plena partida amb el HUD complet (jugador actiu, inventaris, botons de dau, indicador de torn).

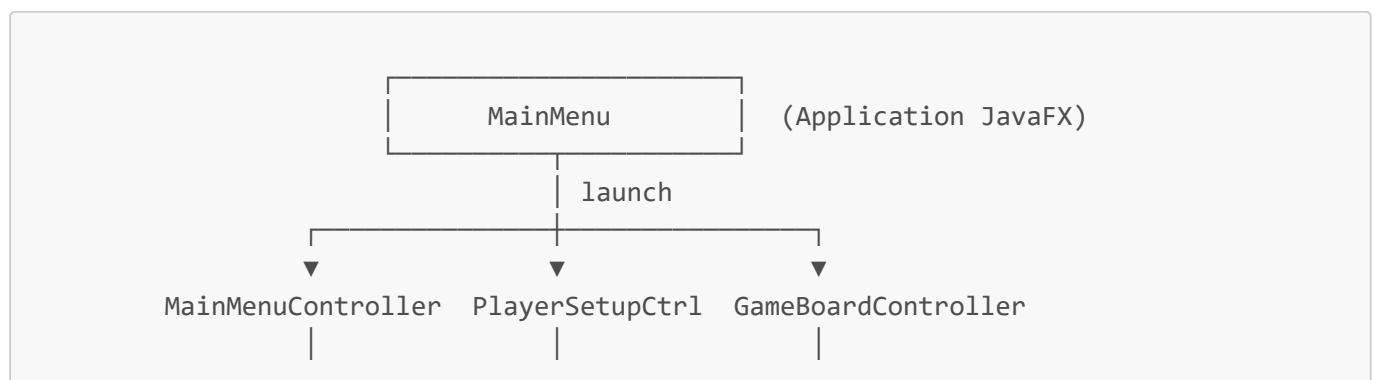
[CAPTURA 5.2] Animació de tirada de dau en marxa amb el valor visible al centre.

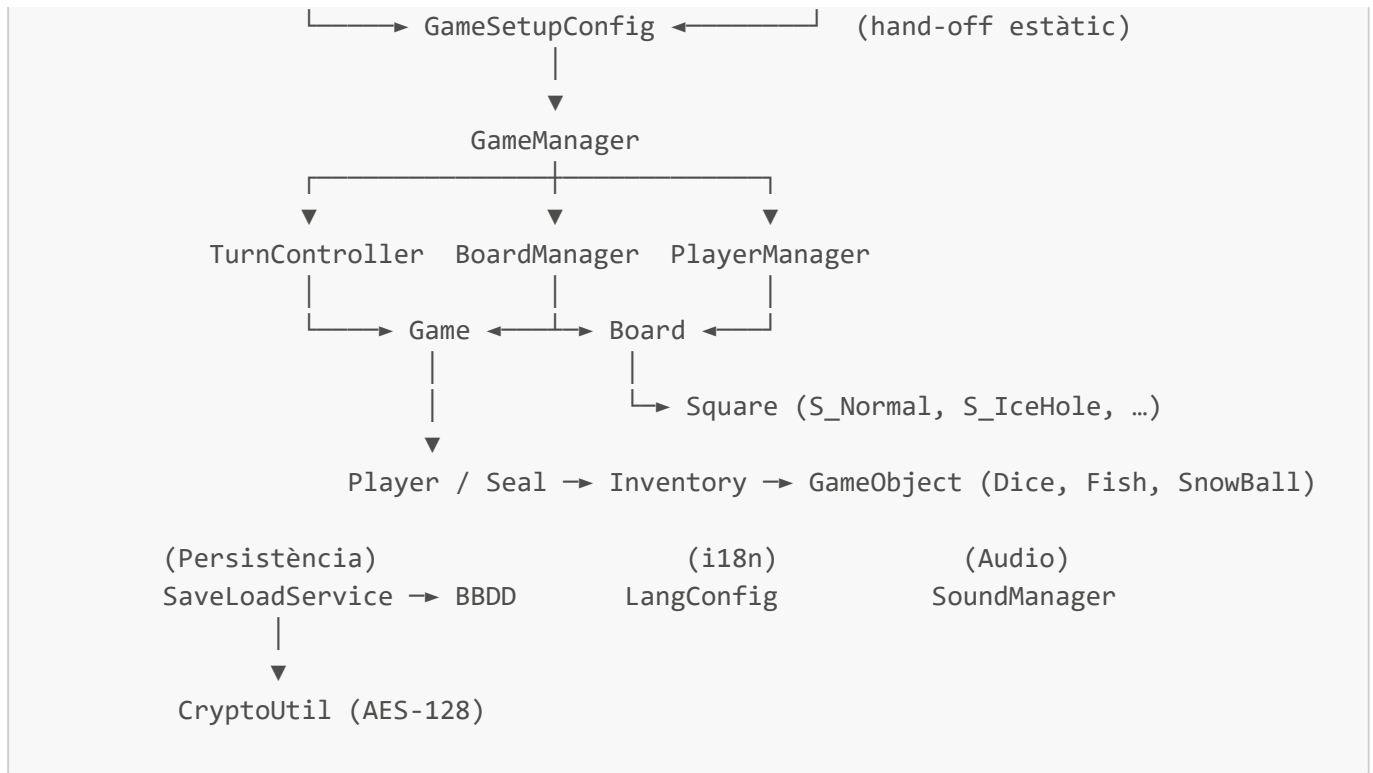
5.5. Flux "Save Game"

1. Usuari prem  **Save Game**.
2. Es demana un nom per a la partida.
3. `SaveLoadService.saveGame()` serialitza l'estat a `YAML`.
4. S'encrpta amb `AES-128` → text `Base64`.
5. `INSERT` a `SAVED_GAMES` via `PreparedStatement` (`CLOB`).
6. Confirmació visual a l'usuari.

6. Relacions entre classes

6.1. Diagrama lògic simplificat





6.2. Relacions importants

Relació	Tipus	Descripció
GameManager → Game	Composició	Manager posseeix l'estat agregat
Game → Board	Composició	Una partida té un tauler
Board → Square[]	Composició	Tauler conté 50 caselles
Square ← S_Normal/S_IceHole/...	Herència	Polimorfisme per tipus de casella
Player extends Entity	Herència	Player i Seal comparteixen comportament base
Player → Inventory	Composició	Cada jugador té un inventari
SaveLoadService → BBDD	Dependència	Persistència delega a la facade JDBC
Controllers → GameSetupConfig	Hand-off	Comunicació estàtica entre pantalles
LangConfig → Listeners	Observer	Canvi d'idioma en runtime notifica tots els controladors

7. Decisions de disseny rellevants

7.1. Per què MVC amb FXML?

FXML separa l'estructura visual (declarativa, en XML) de la lògica (Java). Permet als dissenyadors modificar les pantalles sense tocar el codi i evita els típics blocs grandiloqüents de construcció manual de la UI.

7.2. Per què un MERGE per registrar jugadors?

Si féssim INSERT directe, registrar el mateix nom dos cops llançaria una excepció. MERGE permet ser **idempotent**: la mateixa operació serveix per crear i per actualitzar. Això simplifica molt el codi del controlador (no cal preguntar abans "existeix aquest jugador?").

7.3. Per què PreparedStatement només a saveGame()?

Per a la majoria de sentències, el risc d'injecció SQL és baix (validem els camps abans). Però `GAME_DATA` conté text encriptat amb caràcters arbitraris i sovint > 4000 caràcters, que és el límit d'un literal SQL a Oracle. El PreparedStatement evita l'`ORA-01704` i de pas els problemes d'escapat.

7.4. Per què AES amb clau fixa?

És obfuscació, no seguretat. El joc és local i la BBDD és per al curs: l'objectiu és que un usuari curiós que obri SQL Developer no pugui llegir directament les contrasenyes o l'estat de la partida. Per a producció caldria PBKDF2 + bcrypt + clau no-hardcoded.

7.5. Per què YAML i no JSON o serialització binària?

YAML és llegible per humans (útil per fer debug d'una partida guardada abans d'encriptar) i SnakeYAML mapeja directament `Map<String, Object>` sense necessitat de POJOs ni anotacions, cosa que ens permet evolucionar el format sense reescriure les classes del model.

7.6. Per què el patró Singleton a SoundManager?

Hem de poder canviar de música entre escenes sense que cada controlador hagi de passar-se referències. Un singleton ofereix un punt d'accés global net.

7.7. Per què GameSetupConfig amb camps estàtics?

JavaFX instancia els controladors automàticament via `FXMLLoader`, cosa que fa difícil passar paràmetres entre ells. `GameSetupConfig` actua com a "calaix" estàtic per al hand-off (jugadors, opcions, estat carregat). És pragmàtic en aquest context, encara que pugui semblar un anti-patró en aplicacions més grans.

7.8. Per què caselles encadenades amb llistes auxiliars?

Trineus i forats de gel envien el jugador "al següent" o "a l'anterior" d'aquest tipus. Mantenir dues llistes ordenades d'índexs (`IceHole_Array`, `Sled_Array`) fa que la resolució sigui $O(\log n)$ en comptes d' $O(n)$ i evita haver de recórrer tot el tauler cada vegada.

7.9. Per què Canvas i no ImageView per a les caselles?

El joc és pixel-art: amb `ImageView` i interpolació estàndard de JavaFX, els sprites es borrejaven al redimensionar. `Canvas` amb `setImageSmoothing(false)` i interpolació nearest-neighbour preserva l'aspecte original.

7.10. Per què un mode debug ocult (Ctrl+Shift+D)?

Per accelerar el testing: permet teletransportar pingüins arrossegant-los, forçar el resultat del següent dau i editar inventaris en viu, sense reiniciar la partida.

[CAPTURA 7.1] Mode debug activat amb el panell negre superior, comboBox de jugadors i botons per modificar inventari.

8. Instruccions per compilar i executar

8.1. Requisits previs

- **JDK 17 o superior** (Oracle JDK o OpenJDK).
- **JavaFX SDK 21+** (inclòs a `javafx-lib/`).
- **Driver Oracle JDBC** (`ojdbc8.jar` o `ojdbc11.jar`).
- **SnakeYAML** (`snakeyaml-2.x.jar`).
- **Eclipse IDE** (recomanat) o qualsevol IDE Java amb suport JavaFX.
- Connectivitat amb el servidor Oracle de l'institut (xarxa Ilerna o VPN).

8.2. Importar a Eclipse

1. Obre Eclipse i selecciona *File → Import → Existing Projects into Workspace*.
2. Indica la ruta `eclipse-workspace/JocDelPingu`.
3. Confirma que el `.classpath` i `.project` es detecten.
4. Revisa que les llibreries de `javafx-lib/` estiguin enllaçades al Build Path.
5. Comprova que el JRE configurat sigui Java 17+.

[CAPTURA 8.1] Eclipse amb el projecte JocDelPingu importat i la jerarquia de paquets visible.

8.3. Configuració de l'entorn de BBDD

La connexió es configura a `model/db/BBDD.java`, a la línia 27:

```
String entorno = "fuera"; // canvia a "centro" si treballes des d'Ilerna
```

- **"fuera"**: utilitza `oracle.ilerna.com:1521/XEPDB2`
- **"centro"**: utilitza `192.168.3.26:1521/XEPDB2`

Les credencials estan hardcoded al mateix fitxer (línies 37-41). Per raons de seguretat **no es reproduïxen en aquesta documentació**; consulta el codi font o demana-les al responsable del grup.

8.4. Executar el joc

1. Obre la classe `controller.main.MainMenu`.
2. Botó dret → *Run As → Java Application*.
3. Si Eclipse demana arguments VM, afegeix:

```
--module-path "javafx-lib/lib" --add-modules  
javafx.controls,javafx.fxml,javafx.media,javafx.swing
```

4. S'obrirà la finestra del joc.

8.5. Executar el panell de proves CRUD

1. Obre `view/ui/BBDDPanel.java`.
2. Botó dret → *Run As* → *Java Application*.
3. La sortida apareixerà per consola, executant la seqüència INSERT → SELECT → UPDATE → SELECT → DELETE → SELECT.

8.6. Estructura de sortida

Després de compilar, els `.class` es generen a `bin/` mantenint la jerarquia de paquets. Els recursos d'`assets/` també es copien a `bin/assets/` automàticament per Eclipse.

 **Problema comú:** si en executar veus "Resource not found: /assets/...", revisa que la carpeta `src/assets/` estigui marcada com a *Source folder* al Build Path.